

Active Semantic SLAM: Autonomous mobile robot in an unknown 3D Maze

82523784 Paul Keyvan Hoang-Long Pham Dang

GitHub repo : <https://github.com/ignithor/PyBullet-SLAM-Explorer-Autonomous-robot-in-3D-Maze>

Abstract— Autonomous navigation in unknown environments requires the seamless integration of mapping, localization, and semantic perception. This report paper presents the implementation of a modular autonomous navigation stack for a differential-drive robot in a simulated 3D maze. To address the challenge of localization, we implemented and compared two probabilistic techniques: a Particle Filter (Monte Carlo Localization) and an Extended Kalman Filter (EKF). Furthermore, the system employs a frontier-based exploration strategy driven by Directional A* planning to autonomously map the environment. Finally, the system integrates the CLIP vision-language model to perform zero-shot semantic identification of targets, enabling a complete "search-and-return" mission profile. Experimental results in PyBullet demonstrate the agent's ability to recover from collisions and successfully return to its origin after locating semantic targets.

I. INTRODUCTION

The primary goal of this project is to develop a comprehensive software simulation of an autonomous mobile robot capable of navigating a 3D maze environment. Built using Python and the PyBullet physics engine, the simulation requires the robot to operate in an initially unknown environment, mandating the implementation of autonomous exploration and mapping strategies to construct an internal representation of the world.

To achieve autonomy, the system integrates four distinct technical pillars into a single modular architecture:

- **Localization and Mapping (SLAM):** The robot must utilize a SLAM-inspired technique to simultaneously build a map of the maze and estimate its own position relative to the starting point.
- **Perception and Object Recognition:** Leveraging simulated camera and LiDAR sensors, the robot must apply computer vision techniques to detect and recognize specific target objects defined by the user.
- **Exploration:** The robot must implement algorithms to efficiently explore unknown areas of the maze without prior knowledge of the layout.
- **Path Planning:** After locating the target object, the system must compute the shortest and most efficient path to

return to the origin using the map constructed during the exploration phase.

The software is designed with a modular architecture, distinctively separating the logic for perception, mapping, path planning, and control. This integration combines classical robotics concepts with machine learning techniques to create a fully functional agent capable of completing a "search and return" mission autonomously. The aim of the project is to implement that task from scratch using only simple libraries such as NumPy for the active SLAM part. However, for the perception part, we used the libraries : opencv, transformers and Pytorch.

II. RELATED WORK

The challenge of autonomous navigation has been extensively studied, typically requiring a synthesis of probabilistic robotics and control theory.

A. Simulation Environments

While high-fidelity simulators like Gazebo or Isaac-SIM are standard in industrial robotics, lightweight physics engines such as PyBullet [1] offer a distinct advantage for algorithmic development, allowing for faster-than-real-time training and testing of logic without the overhead of a full middleware stack.

B. Active Exploration and Mapping

Classic approaches often rely on passive teleoperation. However, autonomous agents require Active SLAM, where the robot makes control decisions to minimize map uncertainty. Frontier-based exploration [2] remains the gold standard, directing the robot to boundaries between open space and unexplored territory.

C. Semantic Perception

Traditional object detection relied on closed-set classifiers (e.g., YOLO). The advent of Vision-Language Models (VLMs) like CLIP [3] allows for zero-shot detection, enabling robots to identify objects based on natural language descriptions (e.g., "a yellow duck") without retraining.

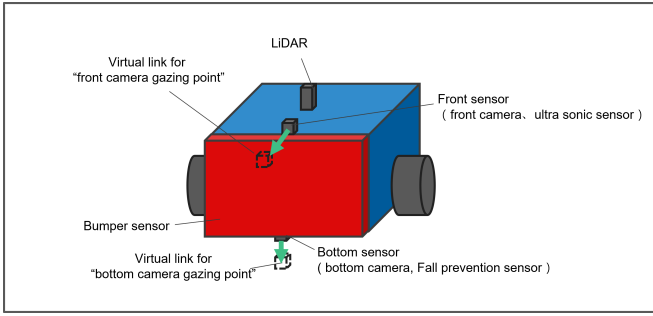


Figure 1: Our mobile robot model

III. SYSTEM ARCHITECTURE

The system architecture follows a modular design ensuring separation of concerns between perception, control, and mapping.

A. The Robot

The agent is a differential-drive mobile robot defined by a URDF model as we can see in Figure 1. For our experiments, we used:

- **Actuation:** Two independent differential-controlled motors.
- **LiDAR:** A simulated 2D scanner emitting 36 rays over a 360-degree field of view with a range of 10.0 meters.
- **Camera:** An onboard RGB sensor capturing 320×240 pixel images, used exclusively for semantic object recognition.

B. Procedural Environment Generation

To ensure robustness, the environment is procedurally generated using a *Recursive Backtracking* algorithm. This guarantees a "perfect" maze (no loops, fully connected) for every run. The generation process follows a depth-first search (DFS) approach on a graph where cells are nodes and walls are edges. Starting from a random seed, the algorithm carves passages to unvisited neighbors until no unvisited neighbors remain, at which point it backtracks. This topology is particularly challenging for frontier exploration algorithms, as it forces the robot to frequently backtrack deep into previously explored branches to find new frontiers, thoroughly testing the path planner's efficiency.

Also, we put three 3D objects : A yellow duck, a soccer ball and a Teddy Bear in the maze. The aim of the robot is to find one of the three object using computer vision technique. In our experiments, the robot has to find the yellow duck. Finding the 2 other objects are useless. We can see a top view of the simulated environment in Figure 2.

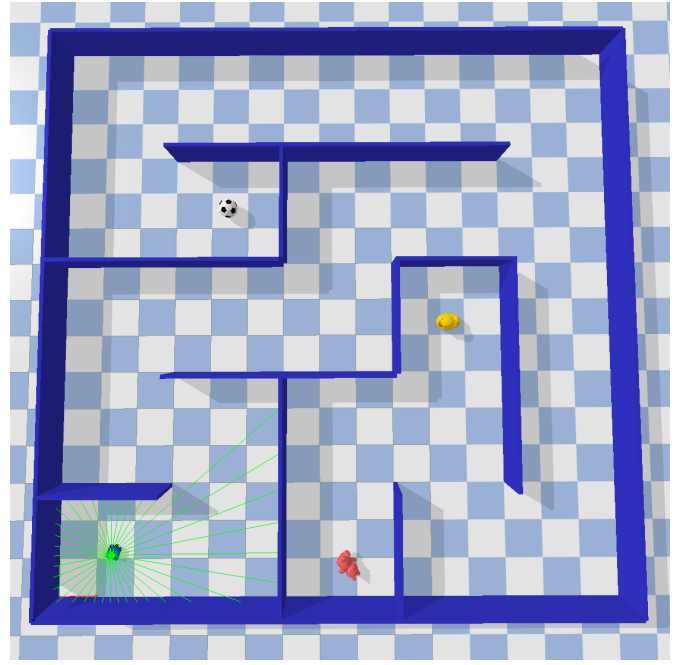


Figure 2: Top view of the PyBullet simulation with the 3D maze, the mobile robot and the three objects.

C. Software Implementation

The simulation framework is built entirely in Python, comprising approximately 2000 lines of code, organized into a modular architecture to separate concerns. The codebase consists of the following key files:

- **config.py:** A centralized configuration file defining system hyperparameters (e.g., PID gains, map resolution, sensor noise) to ensure consistent tuning across modules.
- **control_module.py:** Provides the interface for manual robot control, used primarily for debugging and initial sensor testing.
- **ekf.py:** Implements the Extended Kalman Filter, fusing wheel odometry with compass data to produce smooth trajectory estimates.
- **exploration.py:** Encapsulates the Finite State Machine (FSM), the frontier detection logic, and the adaptive Pure Pursuit controller for path following.
- **main.py:** The entry point script that initializes the simulation environment and triggers the main execution loop.
- **mapping.py:** Handles the Occupancy Grid logic, updating the probability map using ray-casting and log-odds updates.
- **maze_generator.py:** Contains the recursive backtracking algorithm to procedurally generate valid 3D maze environments.

- **particle.filter.py:** Implements Monte Carlo Localization (MCL), using a cloud of particles to robustly estimate global pose under non-Gaussian noise.
- **path_planner.py:** Implements the Directional A* algorithm, computing kinematically feasible paths on the occupancy grid.
- **perception_module.py:** Wraps the CLIP vision-language model, handling image preprocessing and text-image inference for object detection.
- **raw_odometry.py:** Computes dead-reckoning estimates solely from wheel encoders, serving as a baseline for evaluating localization drift.
- **robot.py:** Interfaces with the PyBullet physics engine to manage motor commands and retrieve raw sensor data (LiDAR, Camera).
- **simulation_manager.py:** The central orchestrator class that synchronizes the physics step, sensor acquisition, and the decision-making loop.
- **simple_two_wheel_car.urdf:** The XML description file defining the robot's physical properties (mass, friction, collision shapes) and kinematics.

IV. METHODOLOGY AND IMPLEMENTATION

A. Occupancy Grid Mapping

We implemented a custom SLAM module based on Occupancy Grid Mapping. The environment is discretized into a $0.1m$ resolution grid. The map update rule follows the log-odds formulation:

$$L(m_i|z_{1:t}) = L(m_i|z_{1:t-1}) + \text{inv_sensor}(m_i, z_t) \quad (1)$$

To handle sensor noise and aliasing, we tuned the probabilistic weights to make walls "sticky" ($\text{LogOdds}_{hit} = 0.9$) while free space is learned conservatively ($\text{LogOdds}_{free} = 0.1$). This asymmetry prevents walls from disappearing due to transient ray-casting errors or sensor noise.

B. Probabilistic Localization

To address the drift inherent in dead-reckoning odometry, we implemented and evaluated two distinct probabilistic filters. Each filter processes wheel odometry as a prediction step and compass (IMU) data as a correction step.

1. **Particle Filter (MCL):** A set of 100 particles approximates the posterior distribution of the robot's pose.
 - *Prediction:* Each particle propagates using the differential drive motion model with additive Gaussian noise.
 - *Correction:* Particles are weighted based on the likelihood of their orientation matching the simulated compass.

- *Resampling:* Low Variance Resampling is applied when the effective sample size drops, preventing particle depletion.

2. **Extended Kalman Filter (EKF):** The EKF maintains a Gaussian belief distribution (μ, Σ) . It linearizes the non-linear differential drive motion model using the Jacobian matrix G_t :

$$G_t = \begin{bmatrix} 1 & 0 & -v \sin(\theta) \Delta t \\ 0 & 1 & v \cos(\theta) \Delta t \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

This filter fuses high-frequency wheel odometry with compass data to produce a smooth trajectory estimate.

Additionally, a "stuck detection" logic based on simulated IMU data gates the odometry update to prevent drift when the robot pushes against walls (wheel slip).

C. Active Exploration and Navigation

The robot's decision-making is governed by a Finite State Machine (FSM) comprising four distinct states:

- **EXPLORE:** The default state where the robot autonomously navigates to frontier zones to expand the map. The perception module runs periodically (every 0.5 second) to scan the visual scene for semantic targets.
- **STOP:** Triggered immediately upon confirming the semantic detection of the "yellow duck" (confidence > 0.6). The robot halts all motion to stabilize the map, and registers the target's estimated metric coordinates using LiDAR fusion.
- **PLAN:** The robot computes a global path from its current estimated pose back to the starting origin using the occupancy grid built during the exploration phase.
- **RETURN:** The robot executes the computed path using the adaptive controller. In this mode, the robot focuses solely on path following until it reaches the start zone ($d < 0.4m$).

C.1 Frontier Exploration Theory

We employ a *Frontier-Based Exploration* strategy. A frontier is formally defined as the boundary between known free space ($\mathcal{C}_{free}$) and unknown space ($\mathcal{C}_{unknown}$):

$$\mathcal{F} = \{c \in \mathcal{C}_{free} \mid \exists n \in \text{neighbors}(c), n \in \mathcal{C}_{unknown}\} \quad (3)$$

To maximize exploration efficiency, we utilize a Nearest Frontier heuristic. The robot identifies all valid frontier pixels and selects the target c that minimizes the Euclidean distance from the robot's current pose:

$$\text{Target} = \underset{c \in \mathcal{F}}{\text{argmin}} \|\text{Pose}_{robot} - c\|_2 \quad (4)$$

We also implemented Configuration Space (C-Space) inflation, dilating obstacles by the robot's radius ($R = 0.45m$) to ensure selected frontiers are physically reachable. We also increased the dilatation to $R = 0.8m$ to ensure the robot to not get too close to walls to prevent to get stuck.

C.2 Directional Path Planning

Standard A* algorithms treat robots as holonomic points, often resulting in paths that differential drive robots cannot follow without stopping to pivot. We implemented a Directional A (Hybrid A* Lite) algorithm. The state space is augmented from (x, y) to (x, y, θ) , where θ is discretized into 8 cardinal directions. The cost function $f(n)$ includes a heavy penalty C_{turn} for changes in orientation:

$$f(n) = g(n) + h(n) + C_{turn} \cdot |\Delta\theta| \quad (5)$$

This forces the planner to prefer smooth, forward-moving curves over sharp turns, generating trajectories that are kinematically feasible for the robot to follow at speed.

C.3 Adaptive Control

Path following is handled by an Adaptive Pure Pursuit controller coupled with a PID for angular velocity. The lookahead distance dynamically adjusts based on tracking error ($0.4m$ for curves, $1.0m$ for straights), ensuring precise wall following without clipping corners.

C.4 Fault Tolerance and Recovery

A critical challenge is handling "stuck" scenarios where the robot is physically constrained but the wheels continue to spin. We implemented a *Slip Detection System* utilizing a simulated IMU. The system monitors the discrepancy between commanded velocity and actual body acceleration. If a stuck condition is detected for $N = 50$ frames, a **Recovery Mode** triggers, forcing a blind backward velocity to dislodge the chassis.

D. Semantic Perception

We integrated the pre-trained CLIP model to perform zero-shot classification. At a fixed period, the robot captures an image. The model compares the image embedding against text prompts ("floor", "wall", "a yellow duck", "a soccer ball", "a teddy bear"). A confidence threshold of 0.6 triggers the state transition from EXPLORE to RETURN if the right object is detected as we can see in Figure 3.

Object Localization: Upon semantic detection, the system estimates the target's metric position by fusing visual data with LiDAR scans. We isolate the LiDAR rays corresponding to the camera's field of view ($\pm 30^\circ$ frontal sector) and identify the nearest obstacle distance d_{obj} . The target's global coordinates (x_{obj}, y_{obj}) are computed via a polar-to-cartesian transformation relative to the robot's estimated pose (x_r, y_r, θ_r) :

$$x_{obj} = x_r + (d_{offset} + d_{obj}) \cos(\theta_r + \alpha_{ray}) \quad (6)$$

$$y_{obj} = y_r + (d_{offset} + d_{obj}) \sin(\theta_r + \alpha_{ray}) \quad (7)$$

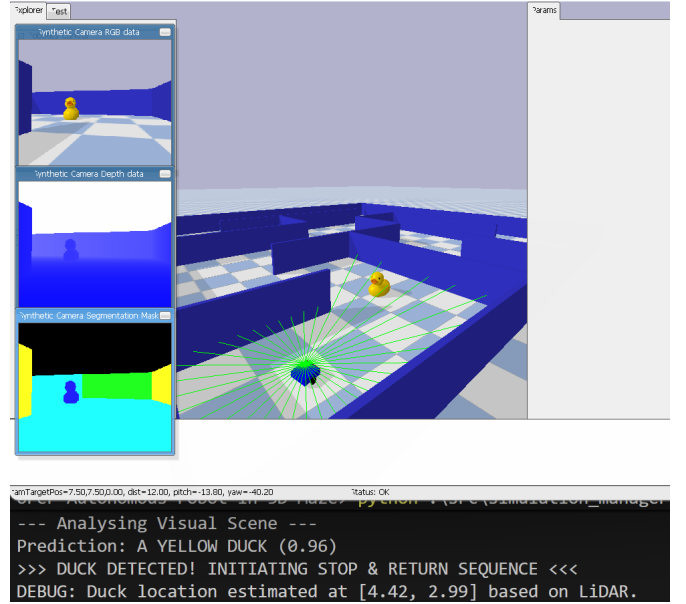


Figure 3: Zero-shot semantic detection of the target object (yellow duck) using the CLIP vision-language model on the robot's RGB camera input.

where d_{offset} accounts for the sensor mounting displacement and α_{ray} is the angle of the ray intersecting the target. An other more precise technique could have been to use the Depth camera of the robot to detect the pose of the object.

V. EXPERIMENTAL RESULTS AND ANALYSIS

The system was evaluated in multiple 10x10 generated mazes. But also works for bigger maze. We tried it first by using perfect localization and on a second hand using two pose estimation methods. I used a laptop with a CPU Intel(R) Core(TM) i7-12700H and Intel(R) Iris(R) Xe Graphics.

A. Exploration Efficiency

The frontier-based strategy successfully mapped the maze area in all trials. The "sticky" log-odds tuning eliminated map flickering, producing sharp, high-contrast maps suitable for planning.

B. Mission Success

The agent successfully distinguished between the target (yellow duck) and distractors (soccer ball, teddy bear) in all the trial. Upon detection, the FSM correctly triggered the Directional A* planner, generating a smooth path back to the origin (0,0). The adaptive controller successfully navigated corridors without getting stuck, utilizing the backup recovery behavior only in rare scenarios when the pose was badly estimated.

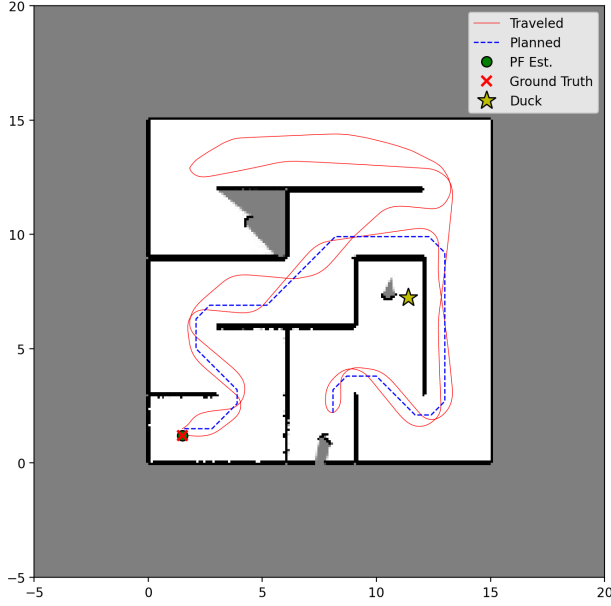


Figure 4: A final map result with the pose duck detected as a star, the pose of the robot and the full trajectory.

C. Parameter Tuning and Sensitivity

Robust performance required extensive tuning of the SLAM and Control subsystems. The angular PID controller was tuned to dampen oscillations on straight paths. We found that a high proportional gain ($K_p = 90.0$) coupled with a moderate derivative term ($K_d = 4.0$) provided the best balance between responsiveness in turns and stability in corridors.

D. Comparative Analysis: EKF vs. PF

To determine the best localization strategy, we analyzed the performance of the EKF and the Particle Filter separately.

- **Particle Filter (MCL):** The PF is theoretically superior for global localization (recovering from being completely lost). However, the discrete resampling step introduces significant high-frequency noise ("jitter") into the position estimate as we can see in Figure 6. This noise caused the derivative term (D) of our PID controller to spike, leading to erratic steering behavior and "wobbling" in straight corridors.
- **Extended Kalman Filter (EKF):** The EKF assumes Gaussian noise and linearizes the motion model. While less capable of recovering from catastrophic localization failures, it produced a much smoother trajectory estimate as we can see in Figure 7. Since the robot starts at a known location $(0, 0, 0)$, the global localization capability of the PF was less critical than the smoothness required for precise path following. Consequently, the EKF was selected as the primary estimator for the final system.

It happens that due to localization error, the robot turn a bit

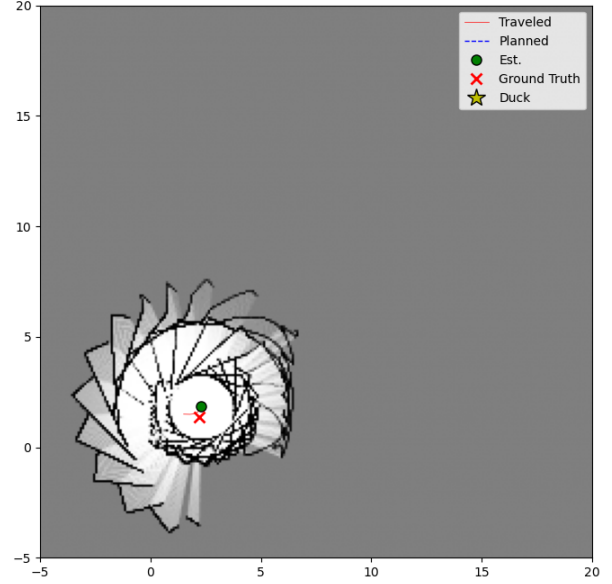


Figure 5: Pose estimation based solely on wheel odometry, highlighting rapid divergence from the true trajectory without a localization filter.

too close toward a wall and get stuck on the wall. Then, the wheels keep turning which lead to a growing error in the localization due to the wheel odometry as they keep turning but the robot is not moving which lead after few seconds the recovery mode. A technique that can improve the pose estimation of the robot could be to do sensor fusion between: the wheel odometry, the IMU, and use visual odometry.

We also try to not use filter and rely only on the wheel odometry for the localization. From the beginning of the simulation, the robot is totally lost and create a total false map. It highlights the fact that a filter is needed for the pose estimation as we can see in figure 5.

On Table 1, we sum up the different results. We use for the metric the average position error in meter from the first 20s of the simulation. As we can see, the EKF technique is by far the most accurate one. We can also notice that error in pose estimation leads to have thicker walls as the LiDAR detects "more" walls due to the uncertainty of the position.

Table 1: Comparison of localization methods based on average error from the first 20s

Method	Avg. Error (m)	Comments
No Filter	∞	Robot gets stuck
PF	1.2178	HF noise & time-derivative effects
EKF	0.1996	Very precise and stable estimation

E. System Latency and Optimization

Real-time performance is constrained by the computational cost of the deep learning perception module.

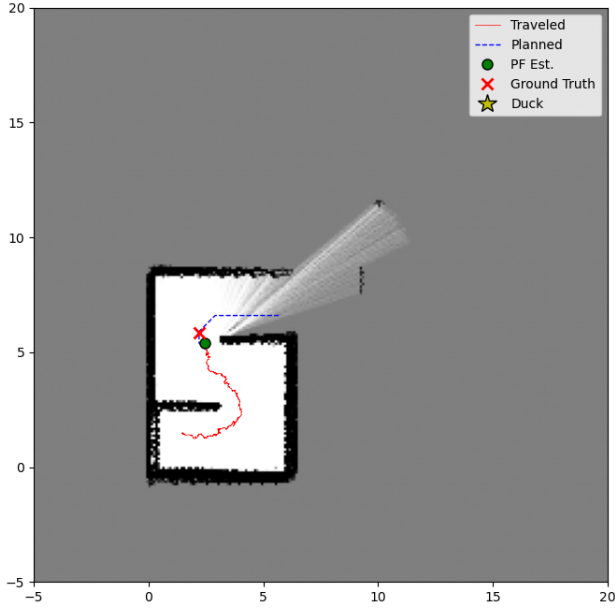


Figure 6: Trajectory estimated using the Particle Filter, showing accurate global localization but noticeable high-frequency noise.

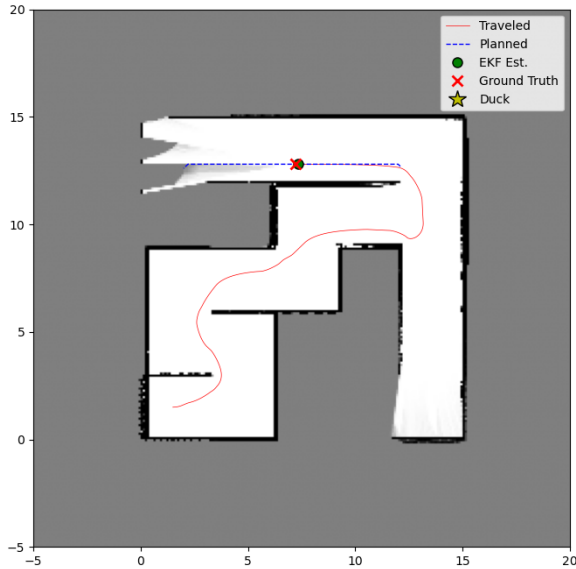


Figure 7: Trajectory estimated using the Extended Kalman Filter, showing smooth and accurate pose estimation with minimal error.

- **Visual Perception (CLIP):** Inference with the *clip-vit-base-patch32* model requires significant GPU/CPU resources. Running detection on every simulation frame (240Hz) caused the physics engine to lag significantly (Real-Time Factor < 0.1). To maintain a fluid simulation, we perform the inference at 2Hz. This triggers the heavy inference task only periodically, balancing the CPU load between physics fidelity and semantic awareness.
- **Map Updates:** Ray-casting 36 rays for SLAM is computationally cheap, allowing for map updates at 150Hz. However, visual plotting via Matplotlib is the primary bottleneck. We throttled the visualization update rate to match the perception interval, ensuring the simulation runs near real-time speeds.

F. The Reality Gap

While PyBullet provides high-fidelity rigid body dynamics, simplifications were made. Our simulation assumes perfect LiDAR rays and a deterministic camera. In a real-world scenario, the CLIP model’s performance would degrade due to motion blur. Furthermore, the wheel friction model in simulation is uniform, whereas real floors exhibit patchy friction. Future physical deployment would require replacing the simulated odometry with visual odometry (VO).

VI. CONCLUSION

This work demonstrated an active semantic SLAM system that enables a mobile robot to autonomously explore an unknown 3D maze, detect a semantic target, and return to its starting position. Frontier-based exploration, directional path planning, and adaptive control allowed reliable navigation in complex environments.

A comparison of localization methods showed that the Extended Kalman Filter provided smoother and more accurate pose estimates than the Particle Filter in this scenario, leading to better control stability. Zero-shot semantic perception using a vision-language model successfully integrated high-level object understanding into the navigation pipeline. Future work might focus on improving robustness through multi-sensor fusion and reducing the simulation-to-reality gap.

REFERENCES

- [1] E. Coumans and Y. Bai, “PyBullet, a Python module for physics simulation,” 2016.
- [2] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997.
- [3] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *ICML*, 2021.
- [4] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.